LAB#2

SORTING AND DIFFERENT TYPES OF OPERATORS IN SQL

OBJECTIVES:

- To learn and implement Arithmetic operatos
- To learn and implement comparison and logical operators in SQL.
- To learn and implement different set operators in SQL (UNION,INTERSECT,EXCEPT).
- To sort the tables by using ORDER by clause.

LAB TASKS:

1. Write a SQL query to display the annual bonus for the employee as an Annual bonus. (Thebonus should be 20% of their salary.

```
SELECT name, salary, salary * 0.20 AS Annual_Bonus FROM Employee;
```

```
name |salary |Annual_Bonus|
-----+-------+------------+
Ali  |50000.0|     10000.0|
Sara |70000.0|     14000.0|
John |90000.0|     18000.0|
Ahmed|60000.0|     12000.0|

4 row(s) fetched.
```

2. Write a SQL query that shows the breakdown of the salary and display the columns asfollows:
   - House Rent (12% of the net salary)
   - Travel Allowances (9% of the net salary)
   - Basic salary (after deducting the allowances from the net salary)
   - Net Salary (the total salary package)
   - Incremented salary (increment the net salary by 10%)

```
SELECT name,salary AS Net_Salary,salary * 0.12 AS House_Rent,salary * 0.09 AS Travel_Allowance,
salary - (salary * 0.12 + salary * 0.09) AS Basic_Salary,
salary * 1.10 AS Incremented_Salary
FROM Employee;
```

```
name |Net_Salary|House_Rent|Travel_Allowance|Basic_Salary|Incremented_Salary|
-----+----------+----------+----------------+------------+------------------+
Ali  |   50000.0|    6000.0|          4500.0|     39500.0| 55000.00000000001|
Sara |   70000.0|    8400.0|          6300.0|     55300.0|           77000.0|
John |   90000.0|   10800.0|          8100.0|     71100.0| 99000.00000000001|
Ahmed|   60000.0|    7200.0|          5400.0|     47400.0|           66000.0|
```

3. Create a database for maintaining student records with the following schema: (Reg_Id, Name, Contact_No, percentage, test_marks, aggregate). Now add another columnwith name admission_status and set values according to their aggregate. If the aggregate isabove 60, set admission_status to Pass or else Fail.

```
CREATE TABLE Student (Reg_Id INTEGER,Name TEXT,Contact_No TEXT,Percentage REAL,
    Test_Marks REAL,
    Aggregate REAL,
    Admission_Status TEXT
);
```

```
> CREATE TABLE Student (Reg_Id INTEGER,Name TEXT,Contact_No TEXT,Percentage REAL,
    Test_Marks REAL,
    Aggregate REAL,
    Admission_Status TEXT
)
```

```
UPDATE Student SET Admission_Status =
CASE
    WHEN Aggregate > 60 THEN 'Pass'
    ELSE 'Fail'
END;
```

```
Reg_Id|Name         |Contact_No |Percentage|Test_Marks|Aggregate|Admission_Status|
------+-------------+-----------+----------+----------+---------+----------------+
     1|Ali Khan     |03001234567|      85.5|      78.0|    81.75|Pass            |
     2|Sara Ahmed   |03111234567|      90.2|      88.5|    89.35|Pass            |
     3|Usman Tariq  |03221234567|      70.0|      65.5|    67.75|Pass            |
     4|Ayesha Malik |03331234567|      88.0|      92.0|     90.0|Pass            |
     5|Bilal Hussain|03441234567|      60.5|      58.0|    59.25|Fail            |
     6|Hina Shah    |03551234567|      75.3|      80.0|    77.65|Pass            |
     7|Fahad Ali    |03661234567|      68.4|      70.0|     69.2|Pass            |
     8|Zainab Noor  |03771234567|      92.5|      95.0|    93.75|Pass            |
     9|Omar Farooq  |03881234567|      55.0|      60.0|     57.5|Fail            |
    10|Iqra Khan    |03991234567|      82.0|      85.0|     83.5|Pass            |
```

4. Write a query to display the names of students who have got above 50% in percentage or test marks.

```
SELECT * FROM Student WHERE Percentage > 50 OR Test_Marks > 50;
```

```
Reg_Id|Name         |Contact_No |Percentage|Test_Marks|Aggregate|Admission_Status|
------+-------------+-----------+----------+----------+---------+----------------+
     1|Ali Khan     |03001234567|      85.5|      78.0|    81.75|Pass            |
     2|Sara Ahmed   |03111234567|      90.2|      88.5|    89.35|Pass            |
     3|Usman Tariq  |03221234567|      70.0|      65.5|    67.75|Pass            |
     4|Ayesha Malik |03331234567|      88.0|      92.0|     90.0|Pass            |
     5|Bilal Hussain|03441234567|      60.5|      58.0|    59.25|Fail            |
     6|Hina Shah    |03551234567|      75.3|      80.0|    77.65|Pass            |
     7|Fahad Ali    |03661234567|      68.4|      70.0|     69.2|Pass            |
     8|Zainab Noor  |03771234567|      92.5|      95.0|    93.75|Pass            |
     9|Omar Farooq  |03881234567|      55.0|      60.0|     57.5|Fail            |
    10|Iqra Khan    |03991234567|      82.0|      85.0|     83.5|Pass            |
```

5. Create a database named Orders. Create a table with the name Local_Orders for customer_orders with the fields' names (Order_No. , Cust_Id, Cust_name, Product_ID,Product_name, Cust_address, branch_address, order_status). Categories for order status(Pending, Processing, Cancelled, Refunded). Now write a query displaying all theresults whose order status is 'Shipped' and 'Delivered'.

```
CREATE TABLE Local_Orders (
    Order_No INTEGER,
    Cust_Id INTEGER,
    Cust_Name TEXT,
    Product_ID INTEGER,
    Product_Name TEXT,
    Cust_Address TEXT,
    Branch_Address TEXT,
    Order_Status TEXT
);
```

```sql
SELECT * FROM Local_Orders WHERE Order_Status IN ('Shipped', 'Delivered');
```

Order_No	Cust_Id	Cust_Name	Product_ID	Product_Name	Cust_Address	Branch_Address	Order_Status
1001	1	Ali Khan	501	Wireless Mouse	Karachi, DHA Phase 5	Karachi, Saddar Branch	Delivered
1002	2	Sara Ahmed	502	Bluetooth Headphones	Karachi, Gulshan-e-Iqbal	Karachi, Gulshan Branch	Shipped
1004	4	Ayesha Malik	504	Laptop Stand	Karachi, Clifton Block 2	Karachi, Clifton Branch	Delivered
1006	6	Hina Raza	506	Smart Watch	Karachi, Bahria Town	Karachi, Bahria Branch	Shipped
1008	8	Zara Noor	508	External Hard Drive	Karachi, Korangi	Karachi, Korangi Branch	Delivered
1009	9	Imran Qureshi	509	Webcam HD	Karachi, Landhi	Karachi, Landhi Branch	Shipped

6. Write a query to display those order numbers which are NOT 'Shipped' OR 'Delivered'.

```sql
SELECT Order_No FROM Local_Orders WHERE Order_Status NOT IN ('Shipped', 'Delivered');
```

Order_No
1003
1005
1007
1010

4 row(s) fetched.

7. Create another table named 'International_Orders' with the same columns as above. NowDisplay all the orders in both tables.

```sql
CREATE TABLE International_Orders AS SELECT * FROM Local_Orders WHERE 1=0;
SELECT * FROM Local_Orders UNION SELECT * FROM International_Orders;
```

Order_No	Cust_Id	Cust_Name	Product_ID	Product_Name	Cust_Address	Branch_Address	Order_Status
1001	1	Ali Khan	501	Wireless Mouse	Karachi, DHA Phase 5	Karachi, Saddar Branch	Delivered
1002	2	Sara Ahmed	502	Bluetooth Headphones	Karachi, Gulshan-e-Iqbal	Karachi, Gulshan Branch	Shipped
1003	3	Usman Sheikh	503	Mechanical Keyboard	Karachi, North Nazimabad	Karachi, Nazimabad Branch	Processing
1004	4	Ayesha Malik	504	Laptop Stand	Karachi, Clifton Block 2	Karachi, Clifton Branch	Delivered
1005	5	Bilal Hussain	505	USB-C Charger	Karachi, PECHS	Karachi, PECHS Branch	Cancelled
1006	6	Hina Raza	506	Smart Watch	Karachi, Bahria Town	Karachi, Bahria Branch	Shipped
1007	7	Fahad Iqbal	507	Gaming Mouse Pad	Karachi, Malir Cantt	Karachi, Malir Branch	Processing
1008	8	Zara Noor	508	External Hard Drive	Karachi, Korangi	Karachi, Korangi Branch	Delivered
1009	9	Imran Qureshi	509	Webcam HD	Karachi, Landhi	Karachi, Landhi Branch	Shipped
1010	10	Nida Farooq	510	Portable Speaker	Karachi, Federal B Area	Karachi, FB Area Branch	Processing

8. Create another table in Employee database with the name attendance_records with the following schema. (employee_id, time_in, time_out, total_time). Now compare and filterout the employee ids from employee_info table to check which employees were present.

```sql
CREATE TABLE Attendance_Records (
    employee_id INTEGER,
    time_in TEXT,
    time_out TEXT,
    total_time TEXT
);
SELECT emp_id FROM Employee INTERSECT SELECT employee_id FROM Attendance_Records;
```

9. Display the employee record order by salary (in asceneding order).

```sql
SELECT * FROM Employee ORDER BY salary ASC;
```

emp_id	name	salary	dept
1	Ali	50000.0	HR
4	Ahmed	60000.0	Sales
2	Sara	70000.0	Sales
3	John	90000.0	Finance

10. Explore the LIKE operator and implement two use cases on the above database.
Printing Names which Start With a By Using A%;

```
SELECT * FROM Employee WHERE name LIKE 'A%';
```

```
emp_id|name |salary |dept |
------+-----+-------+-----+
     1|Ali  |50000.0|HR   |
     4|Ahmed|60000.0|Sales|
```

Printing Names which ends with %a.

```
SELECT * FROM Employee WHERE name LIKE '%a';
```

```
emp_id|name|salary |dept |
------+----+-------+-----+
     2|Sara|70000.0|Sales|
```

11. Answer the following questions:

- What have you learned from the lab task?
  I have Gone through Ar thematic operators Explored Intersection and Union Concatenation via union Conditional Presenting and How Different data tabels can have same attributes or column which we can later on display In addition to that I have explores LIKE keyword in SQL where I printed on the basis of condition.
- What was the most challenging task and how did you overcome that challenge?
  I would say Most Challenging task would not to mess up the arrangements of the keywords although memorized still have trouble to use it in query some time Dummy data pisses me of.